Axiomatic Systems Programming: Structural Type Theory and Exact Arithmetic in C++23

The `dedekind` Authors

April 4, 2026

Abstract

This paper introduces `dedekind`, a C++23 library designed to embed formal mathematical structures directly within the language's type system. We address the performance-productivity gap in computational science—often characterized as the "two-language problem"—by leveraging modern C++ features, including modules, concepts, and non-type template parameters (NTTP). Unlike traditional object-oriented frameworks that rely on nominal inheritance, `dedekind` employs a structural type theory to reify mathematical entities as compile-time specifications. We demonstrate the utility of this approach through a formal construction of the continuum $\mathbb{R}$ via Dedekind cuts, enabling the exact resolution of transcendental numbers without floating-point overhead. Furthermore, we show how hoisting logical invariants into type signatures allows the LLVM backend to perform aggressive structural pruning, such as collapsing contradictory set-builder predicates into zero-instruction machine code. This work illustrates that integrating formal mathematical rigor into systems programming can yield zero-overhead abstractions suitable for high-performance symbolic computing.

1 Introduction

The development of computational science software is often constrained by a persistent performance gap between high-level symbolic environments and low-level execution backends. While tools like `PyTorch` [1] and `NumPy` [2] offer expressive interfaces for symbolic reasoning, they typically rely on C++ backends that treat mathematical logic as a runtime implementation detail. This results in a *two-language problem* where the high-level mathematical intent is decoupled from the underlying hardware execution.

Integrating high-performance C++ with the `Python` ecosystem often involves a trade-off between manual binding effort and runtime overhead. While legacy tools like `ctypes` provide a standard library solution, they lack native C++ type safety and necessitate verbose wrappers. Modern alternatives such as `nanobind` [3] and `cppyy` [4] represent the current frontier: `nanobind` leverages C++17 to provide high-performance bindings with minimal binary bloat, while `cppyy` utilizes Just-In-Time (JIT) compilation to automate the mapping of complex C++ headers. However, these tools introduce specific integration challenges, including sophisticated `CMake` build infrastructures or heavy `LLVM` backends, which can complicate cross-platform distribution.

This paper introduces `dedekind`, a C++23 [5] library that approaches these challenges by a *direct embedding of mathematical concepts* as a domain specific language (DSL) in the C++ language syntax. By utilizing modern C++ features, including `modules`, `concepts`, `constexpr` and `require` and non-type template parameters (`NTTP`), the library enables the structural integrity of a mathematical model to be enforced at compile-time. This way, the C++ compiler is used as both a verification and an optimization engine, resulting in machine code that mirrors the efficiency of hand-optimized assembly. We hope to show that recently added C++23 features—such as modules, `concept`, `consteval`, and non-type template parameters (NTTP)—allow developers to define complex mathematical species using a syntax that mirrors formal logic. Crucially, these are not mere runtime "filter" objects; they are compile-time structural specifications [6]. Just as a C++ concept defines a set of behaviors without mandating a specific memory layout, Structuralism [7] treats mathematical entities as positions in a system of relations.

As a goal, we set ourselves to model powerful abstraction in mathematical logic such as concepts from category theory number theory or set-builder notation. In the latter, an often transfinite collection of objects such as $\{x \in \mathbb{R} \mid \mathbf{P}(x)\}$ is deinfed purely by the selection of a universe (in this case $\mathbb{R}$) and predicates $\mathbf{P}(x)$ which must be matched by the elements of that universe. In traditional systems programming, this declarative elegance is lost in the management of imperative loops, filter functions and temporary buffers. The translation effort in turn is often manual, difficult and error prone. We hope to demonstrate that this need not always be the case.

Correspondingly, the aim of the `dedekind` library is to serve as a high-performance bridge for the computational sciences, specifically aiming to narrow the gap between C++ and the symbolic algebra systems prevalent in the Python ecosystem. While Python allows for the expressive definition of the Continuum, the execution is often decoupled from the hardware's potential. Conversely, while the Rust programming language provides a "safety-first" approach to systems programming, its trait system currently lacks the "white-box" structural transparency required to hoist complex mereological invariants, such as the Dedekind cut, into the heart of the optimization pipeline.

By reifying the Dedekind cut—defining real numbers $\mathbb{R}$ as partitions of the rationals $\mathbb{Q}$ —directly within the C++ type system, we enable the exact representation of transcendental numbers like e and π at compile-time. This methodology allows the C++ compiler to act as a physical guardrail for mathematical truth, ensuring that a "Ring" or a "Field" is not merely a label, but a verified invariant.

However, reifying a formal ontology in a statically-typed **systems** language presents significant challenges compared to "pure" functional languages like Haskell [8] or Agda [9], which have native support for higher-kinded types, dependent types and first-class functions. Challenges encountered include:

- **Template Template Parameters**: C++ is notoriously bad at true Category Theory because it lacks higher-kinded Types (HKTs). As a result `Functor<F>` or `Monad<F>` are defined in terms of `template template`, which have significant edge cases (e.g., they don't play well with aliases or multiple template arguments).

- **The Lambda Hurdle (Opaqueness)**: Unlike the first-class, inspectable functions in the ML family, C++ lambdas are unique, anonymous types. They do not inherently expose their domain or codomain to the type system. A morphism's domain and codomain must be inferred at the call site or explicitly provided by the programmer. Although if used as *Non-Type Template Parameters* (NTTP), they can be "inspected" them via `concept` checks or by wrapping them in a type that carries its own signature metadata.

- **Decidability vs. Instantiation**: While dependent-type systems use proof-search, C++ relies on template instantiation. This introduces the risk of "Late Detection," where a mathematical violation is only discovered when a set is materialized, rather than when it is defined. This can be mitigated by forcing evaluations into `consteval` and `constexpr` contexts.

- **The Performance-Rigidity Tradeoff**: High-level abstractions in C++ often incur a "template tax" in compile times. We utilize C++23 **Module Partitions** to create a directed acyclic graph of truth, caching structural invariants to ensure that formal rigor does not collapse the build pipeline.

- **The Specialization Constraint**: Unlike the unified pattern matching of functional languages, C++ relies on Partial Template Specialization to resolve structural properties.

This creates a "Rigidity Challenge": the compiler cannot readily deduce that a property of A also applies to $A \cup B$ without explicit boilerplate. We address this by using `concept ... requires` as logical compile-time predicate, thereby simulating the polymorphic reach of a true dependent-type system.

- **The Precedence-Associativity Constraint**: A significant friction point in embedding categorical notation within the C++ grammar is the language's fixed set of `operator` candidates and fixed precedence and associativity rules. While the `operator>>=` is used to model the Monadic Bind ($\gg=$), the ISO C++ standard defines compound assignment operators (including `>>=`, `<<=`, `+=`, `*=`, etc.) with **right-to-left associativity**. Consequently, a standard monadic chain such as $m \gg= f \gg= g = (m \gg= f) \gg= g$ is parsed by the compiler as `m >>= (f >>= g)`. As this makes no sense in a monadic chain, explicit grouping, e.g., `(m >>= f) >>= g` is required. As an alternative, we provide `operator>>` ("Highway Notation"), but with slightly different categorical semantics.

We note that while these kinds of restrictions may make C++ appear frugal when compared to a "proper" functional programming environment, the small runtime footprint of C++ will still make the approach worthwhile for many use cases. Here, we also note that the Rust programming language may have a similar thrust as a strongly type-checked systems programming language. However, the aim of the DEDEKIND library is more focussed on serving the high-performance Python ecosystem. While Python's symbolic algebra systems, such as SymPy, offer unparalleled declarative ease, they often act as "slow" oracles that require manual translation into C++ for production performance. DEDEKIND aims to narrow this gap by allowing the C++ compiler itself to understand the symbolic logic of the Continuum.

By implementing the Dedekind cut—defining real numbers $\mathbb{R}$ as partitions of the rationals $\mathbb{Q}$ —directly within the type system, we enable the exact representation of transcendental numbers like e and π without floating-point approximation. This puts dedekind in a unique position relative to Rust: while Rust provides superior memory safety through its borrow checker, C++23's advanced template metaprogramming and modules allow for a deeper, "white-box" formal verification of mathematical structures that Rust's current trait system cannot easily replicate.

1.1 Instrumented Model Validation: The CI Pipeline as a Prover

While formal verification in languages like Agda or Coq typically relies on static type-level proofs that are erased during compilation, the dedekind framework adopts a methodology of **Instrumented Model Validation**. By leveraging the LLVM profiling infrastructure, we provide an empirical *Runtime Witness* for our categorical primitives.

This approach serves as a deterministic evolution of the property-based testing tradition pioneered by QuickCheck [10]. Where QuickCheck utilizes stochastic search to falsify algebraic properties in Haskell, dedekind utilizes the CI/CD pipeline to ensure that every structural identity is physically materialized in the binary. By enforcing 100% patch coverage on new ontological partitions, we provide a "Physical Proof" that the abstract category theory—often treated as a ghost in the machine—is actually traversed by the hardware. This aligns with the "Real World" functional programming philosophy advocated by O'Sullivan et al. [11], where formal rigor is balanced with the practical necessity of observable, high-performance execution.

Despite these constraints, the contributions of this work include:

- **Intensional-to-Extensional Materialization**: A strategy for partial, lazy evaluation, where a set defined by a symbolic rule is only converted into machine instructions (extensional data) when an operation requires it. This enables the evaluation of Infinite Sets (e.g., the Naturals $\mathbb{N}$) within a finite execution environment.

- **Dedekind Cuts**: exact representation of both finite as well as infinite series, enabling the Dedekind cut to define the real numbers $\mathbb{R}$ (including the transcendental numbers $e, \pi, \sqrt{2}$) *exactly*.
- **Complex and Dual Numbers**: *exact* representation of differentiable functions through augmented real number lines. Perhaps we can even prove Euler's formula, but I am not certain.
- **Mereological Pruning of Predicates**: Because the library understands the logical structure of the set-builder predicates, the compiler can prune the search space of the set. For example, a set defined by a contradiction $\{x \in \mathbb{S} \mid \mathbf{P}(x) \wedge \neg P(x)\}$ is collapsed into the empty set $\emptyset$ at compile-time, emitting zero machine instructions.
- **Set-Builder Isomorphism**: A syntax-driven mapping that allows expressions like

  $$\mathrm{Set}\{\mathrm{x} \,\%\, \mathbb{R} \wedge (\mathrm{x} + 5 < 10)\} \tag{1}$$

  to be resolved at compile-time. We demonstrate how the C++ type system acts as a Constraint Satisfaction Engine to verify the validity of these sets.
- **Zero-Overhead Semantic Mapping**: We show that these high-level abstractions result in machine code that is identical to hand-optimized assembly, effectively proving that formal rigor does not necessitate a "performance tax."

1.2 Structural Type Theory in C++23

1.2.1 Nominal vs. Structural Subtyping

Traditional object-oriented systems, such as those implemented in Java or C#, typically rely on *nominal subtyping*. In these systems, a type T is recognized as belonging to a category C only through explicit inheritance declarations (e.g., `class T : public Monoid`). Instead of this taxonomic approach the `dedekind` prefers a structural type theory. This alignment mirrors the mathematical structuralism advocated by Awodey [12], where mathematical objects are defined not by their internal 'essence' but by their roles and relations within a category. Leveraging C++20/23 Concepts, types are treated as mathematical objects defined entirely by their interface and satisfied invariants. A type T is identified as a `SmallCategory` if it possesses the required algebraic morphisms $(\otimes, e)$; its internal name and declaration lineage are irrelevant to the compiler's logical resolution.

1.2.2 The Compiler as a Verification and Optimization Engine

While functional languages like Haskell are often viewed as the natural home for Category Theory, the foundations for a structuralist approach in C++ have been well-established by Milewski [13], who demonstrated that C++'s type system is capable of modeling complex categorical morphisms. `dedekind` builds upon this conceptual bridge by utilizing modern C++23 features to move from mere simulation to compile-time verification and optimization. By applying `static_assert` against structural concepts, we transform the compilation process into a formal verification phase. Crucially, this provides the `LLVM` toolchain with the semantic context required for "structural pruning." By hoisting mereological relations into the type signature, the backend can identify logical contradictions as unreachable code paths. By using `constexpr` and `requires`, to identify and potentially prune complex expressions—such as an empty set defined by contradictory predicates—into zero-overhead machine instructions. This "correctness-by-construction" methodology ensures that the structural integrity of the mathematical model is verified and optimized before the final binary is generated.

1.2.3 The :set partition

To ground this theoretical approach, Table 1 maps the fundamental axioms of the Elementary Theory of the Category of Sets (ETCS) – as codified in the pedagogical treatment by Lawvere and Rosebrugh [14] – to their structural equivalents in dedekind. This mapping ensures that the library is not merely a numerical simulation, but a direct implementation of algebraic laws within the host language's type system. In a sense, dedekind hopes to be a *compiler* for mathematical concepts.

Table 1: Complete Mapping of ETCS Axioms to C++23 Structural Invariants

ETCS Axiom	C++ Language Feature	Structural Invariant
1. Composition	std::invoke / operator»	IsSemigroupoid<T, Op>
2. Identity	identity_trait<T, Op>	IsSmallCategory<T, Op>
3. Terminal Object	s(x) -> Logic::top	IsTerminalObject<T>
4. Well-Pointedness	operator() (Element access)	IsWellPointed
5. Cartesian Product	std::tuple / std::pair	HasProduct<A, B>
6. Exponentiation	Lambda NTTP / std::function	IsInternalHom<A, B>
7. Equalizers	dedekind::set<T, P>	IsEqualizer<f, g>
8. Empty Set	dedekind::EmptySet	IsInitialObject<T>
9. Infinity (NNO)	dedekind::Naturals	HasNaturalNumbersObject
10. Axiom of Choice	Lattice Dispatcher	AxiomOfChoice

By adopting a categorical foundation, the library provides a unified interface for both *extensional* and *intensional* representations. For finite collections, such as the Booleans, the system performs *extensional* materialization to compute exact results at compile-time. Conversely, for infinite or transcendental structures like the Complex Numbers, the library utilizes *intensional* symbolic expressions to maintain structural integrity. By embedding these categorical definitions in the C++ host language, the programmer—and crucially the *compiler*—can examine the algebraic properties of a field even when the underlying elements cannot be fully materialized as discrete machine data.

1.2.4 Intensional Sets and Computational Undecidability

The utility of the categorical approach is most evident when defining sets whose membership conditions are conceptually clear but computationally undecidable. Consider the set of *Normal Numbers* $\mathscr{N}$, defined as those real numbers whose infinite digit expansion follows a uniform distribution. Formally, a real number x is normal in base b if for every finite sequence of digits s, the limiting frequency of s appearing in the expansion of x satisfies:

$$\lim_{n\to\infty} \frac{N(s,x,n)}{n} = \frac{1}{b^{|s|}} \tag{2}$$

where $N(s, x, n)$ denotes the number of occurrences of the string s in the first n digits of x. While we can easily define this property as a symbolic predicate within the dedekind ontology, the membership of specific constants like π or e remains a famous open problem in mathematics.

In a traditional imperative framework, such a set is unusable because membership cannot be *computed* to a boolean result. However, by treating $\mathscr{N}$ as an *intensional* set, the dedekind library allows the programmer to define morphisms that take $\mathscr{N}$ as their formal domain. This enables the C++ compiler to verify the structural validity of a function pipeline based solely on the *definition* of normality, even while the empirical membership of π remains unresolved. By hoisting the predicate into a Stateless Constraint Type, we decouple the logical requirement of the set from the necessity of materializing its elements. This allows the compiler to treat $\mathscr{N}$ as a valid algebraic object, facilitating symbolic reasoning about digit distribution without requiring a prior numerical proof of normality, thereby avoiding the runtime overhead of high-precision digit evaluation.

This allows the compiler to treat $\mathscr{N}$ as a valid algebraic object, facilitating symbolic reasoning about digit distribution without requiring a prior numerical proof of normality, thereby avoiding the runtime overhead of high-precision digit evaluation.

This intensional logic extends to non-elementary functions where the antiderivative cannot be expressed in a finite combination of elementary terms. For example, the Gaussian integral e^{-t^2} defines the *Error Function* (erf) as a structural rule for area rather than a simple algebraic formula:

$$\operatorname{erf}(x) = \frac{2}{\sqrt{\pi}} \int_0^x e^{-t^2} dt \tag{3}$$

While traditional backends rely on lossy polynomial approximations to evaluate this integral, the `dedekind` ontology reifies $\operatorname{erf}(x)$ as a set of symbolic properties. This allows the `C++` compiler to resolve identities—such as the symmetry $\operatorname{erf}(-x) = -\operatorname{erf}(x)$—as zero-overhead type transformations, delaying numerical approximation until the final realization of the continuum.

1.2.5 Categorification of Sequences and the Continuum

While ETCS provides the axioms for discrete collections, the representation of sequences and the analytic continuum requires a transition from set-theoretic membership to functorial structures. In the `dedekind` library, we avoid the imperative indexing of terms by treating sequences as *Combinatorial Species* [15]. By defining a sequence as a functor from the category of finite sets to itself, we can express growth rules—such as the Fibonacci recurrence—as algebraic identities between functors. This allows the compiler to resolve the structural identity of a series through symbolic substitution before any machine-level summation occurs [16]. To reach the exact real arithmetic promised in Section 1, we ground our implementation of convergence in LAWVERE's theory of enriched categories [17]. By treating metric spaces as categories enriched over the poset of non-negative reals $[0, \infty)$, the library reifies Cauchy sequences not as floating-point approximations, but as formal Cauchy completions [18]. This methodology ensures that the *Dedekind cut*—defining real numbers $\mathbb{R}$ as partitions of the rationals $\mathbb{Q}$ [14]—is not merely a runtime filter, but a verified limit of a rational sequence. This allows the `LLVM` backend to treat transcendental constants like e and π as symbolic invariants rather than opaque numeric variables.

The library addresses the representation of transcendental constants by distinguishing between their algebraic *definition* and their numerical *realization.* While the machine may be unable to *compute* the final digit of π, the `dedekind` ontology can lazily *define* it as a formal limit of a rational sequence. By encoding the LEIBNIZ series for π or the TAYLOR expansion for e, the library treats these constants as symbolic *intensional* types.

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + 1 + \frac{1}{2} + \frac{1}{6} + \dots \tag{4}$$

This approach leverages *lazy evaluation* at the type-level, where the `C++` compiler performs symbolic substitutions of the series terms before any floating-point approximation is required. By teaching the compiler the *rule* of the sequence rather than a static value, the library ensures that mathematical identities—such as $e^0 = 1$—can be resolved as zero-instruction compile-time proofs.

xxxxxx

While ETCS provides the axioms for discrete collections, the representation of sequences and the analytic continuum requires a transition from set-theoretic membership to functorial structures. In the `dedekind` library, we avoid the imperative indexing of terms by treating sequences as Combinatorial Species [15]. By defining a sequence as a functor from the category of

Table 2: Categorification of Sequences and Series in the Dedekind Ontology

Mathematical Concept	Set-Theoretic Form	Categorical Framework	C++ Structural
Recursive Sequence	$F_n = F_{n-1} + F_{n-2}$	Combinatorial Species [15]	Functorial Coprodu
Formal Power Series	$\sum a_n x^n$	Analytic Functors [16]	Polynomial Species
Metric/Distance	$\|x - y\| < \epsilon$	Lawvere Enriched Category [17]	Enriched Hom-sets
Cauchy Convergence	$\lim_{n\to\infty} a_n$	Cauchy Completion [18]	Limit / Terminal O
Real Numbers ($\mathbb{R}$)	Dedekind Cuts / Cauchy	Archimedean Completion [19]	Adjoint Functors

finite sets to itself, we can express growth rules—such as the Fibonacci recurrence—as algebraic identities between functors. This allows the compiler to resolve the "combinatorial essence" of a series through symbolic substitution before any machine-level summation occurs [16]. To reach the exact real arithmetic promised in Section 1, we ground our implementation of convergence in Lawvere's theory of enriched categories [17]. By treating metric spaces as categories enriched over the poset of non-negative reals $[0, \infty)$, the library reifies Cauchy sequences not as floating-point approximations, but as formal Cauchy completions [18]. This methodology ensures that the Dedekind cut – defining real numbers as partitions of the rationals [14] – is not merely a runtime filter, but a verified limit of a rational sequence, allowing the LLVM backend to treat transcendental constants like e and π as first-class structural invariants.

The library addresses the representation of transcendental constants by distinguishing between their algebraic *definition* and their numerical *realization*. While the machine may be unable to *compute* the final digit of π, the `dedekind` ontology can lazily *define* it as a formal limit of a rational sequence. By encoding the Leibniz series for π or the Taylor expansion for e, the library treats these constants as symbolic *intensional* types.

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} = 1 + 1 + \frac{1}{2} + \frac{1}{6} + \dots \tag{5}$$

This approach leverages `lazy evaluation` at the type-level, where the `C++` compiler performs symbolic substitutions of the series terms before any floating-point approximation is required. By teaching the compiler the *rule* of the sequence rather than a static value, the library ensures that mathematical identities—such as $e^0 = 1$—can be resolved as zero-instruction compile-time proofs.

Table 3: Categorification of Sequences and Series in the Dedekind Ontology

Mathematical Concept	Set-Theoretic Form	Categorical Framework	C++ Structural
Recursive Sequence	$F_n = F_{n-1} + F_{n-2}$	Combinatorial Species [15]	Functorial Coprodu
Formal Power Series	$\sum a_n x^n$	Analytic Functors [16]	Polynomial Species
Metric/Distance	$\|x - y\| < \epsilon$	Lawvere Enriched Category [17]	Enriched Hom-sets
Cauchy Convergence	$\lim_{n\to\infty} a_n$	Cauchy Completion [18]	Limit / Terminal O
Real Numbers ($\mathbb{R}$)	Dedekind Cuts / Cauchy	Archimedean Completion [19]	Adjoint Functors

This mapping ensures that our software implementation is not merely a *simulation* of set theory, but a direct *instantiation* of its algebraic laws.

The structural mereology [20] approach in `dedekind` defines sets via formal relations—union $(X \cup Y)$, intersection $(X \cap Y)$, and parthood $(X \subseteq Y)$ and last but not least element $(x \in Y)$ – prior to implementation. This mirrors structuralist foundations such as Lawvere's Elementary Theory of the Category of Sets (ETCS) [21] and Cartesian Closed Categories (CCCs) [22, 23]. By prioritizing *structural transparency*, this methodology fundamentally departs from object-oriented encapsulation. In a sense, it encodes the well known rule of thumb of "composition over inheritance" [24]. In a standard `std::set`, the internal representation is an opaque implementation detail, invisible to the type system's logic. Conversely, in `dedekind`, the *type is the definition*. Because the set's constituents—predicates, universes, and mereological relations—are hoisted into the type signature as first-class members, the C++ type system and the `CMake` toolchain function as a Constraint Satisfaction Engine (CSE). This allows the compiler to inspect, decompose, and optimize a set based on its logical essence rather than its storage layout, trading the 'black-box' safety of encapsulation for the 'white-box' provability of formal mereology.

```
#include <concepts>
#include <type_traits>

// Verifies the "Element Of" relation (x in Y)
template <typename T, typename S>
concept IsElementOf = requires(T x, S s) {
    { s.contains(x) } -> std::same_as<bool>;
};

// Verifies "Parthood" or "Subset" relation (X subset of Y)
template <typename Sub, typename Super>
concept IsSubsetOf = requires {
    requires std::derived_from<Sub, Super> ||
             requires { typename Sub::is_subset_of<Super>; };
};

// CSE Pruning: Collapsing contradictory intersections to EmptySet
template <typename SetA, typename SetB>
struct Intersection {
    using type = std::conditional_t<
        HasContradictoryPredicates_v<SetA, SetB>, // Evaluated at compile-time
        EmptySet,
        InternalIntersection<SetA, SetB>
    >;
};
```

Listing 1: Compile-time mereological verification using C++23 concepts.

By reifying a subset of the set-builder notation, dedekind turns the C++ translation unit into a symbolic laboratory. The programmer is free to reason in the language of Cantor, while the compiler produces the performance of a naked C++ loop.

2 Implementation

The methodology of `dedekind` is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next. The intention being that the flow of compilation would mirror the order in which objects might get introduced in a textbook or a lecture. I.e. objects which are compiled first would tend to be more general, more foundational (either in the mathematical sense or in terms of embedding in the C++ host language) whereas objects which get compiled later will be more elaborate and specialized but also perhaps more practical to the user.

To ensure the decidability of the ontology and mitigate the 'template tax,' we utilize a strictly enforced build graph (see Figure 1). This DAG mirrors the formal structure of a mathematical proof: *Axiom* $\rightarrow$ *Theorem* $\rightarrow$ *Proof*. By isolating foundational mereological axioms into the `:mereology` partition, the compiler produces a Binary Module Interface (BMI) that acts as a 'cached truth.' This physical stratification prohibits circular reasoning at the compiler level; a partition can only reference previously verified structural properties. Subsequent partitions, such as `:algebra`, import these BMIs as established lemmas, allowing the compiler to focus solely on the synthesis of higher-order laws without the risk of recursive definition or re-instantiation of the underlying species. In this architecture, the C++ build system serves as a physical guardrail for the integrity of the formal derivation.

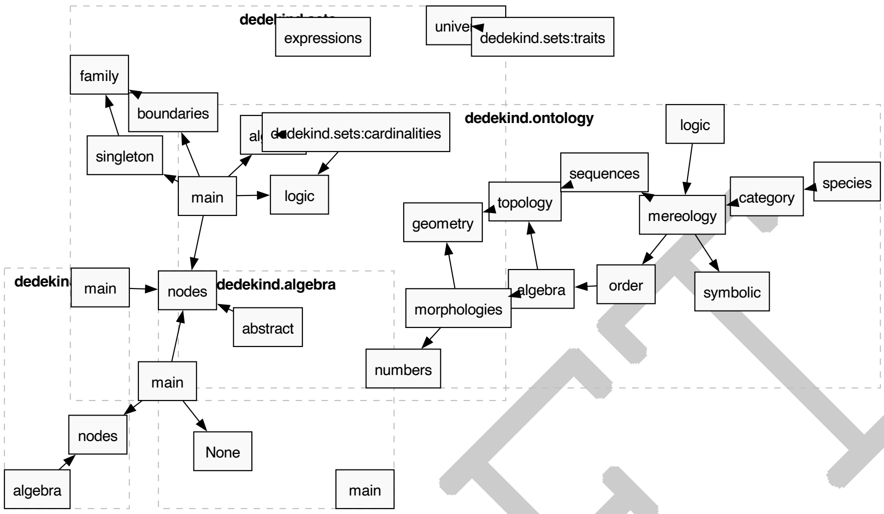

Figure 1: The Ontological Build Graph: Automated dependency resolution.

2.1 `category` Module

The methodology of `dedekind` is predicated on the translation of formal mathematical structures into a stratified registry of C++23 modules. This stratification allows for a "bottom-up" synthesis of complex mathematical objects, where each level serves as a verified foundation for the next. The intention is that the flow of compilation mirrors the order in which objects are introduced in a formal textbook: foundational mereological axioms are isolated into the `:mereology` partition, which produces a Binary Module Interface (BMI) that acts as a "cached truth."

2.1.1 The `:species` partition

The `:species` partition functions as an *Algebraic Atlas*, providing a stratified registry of structural invariants for C++ primitive types and standard library operators. By reifying these categorical axioms—associativity, commutativity, and idempotency—as compile-time constants, the library transforms the C++ type system into a formal verification engine. This design follows the combinatorial species theory of Joyal [15, 16], treating fundamental types not merely as sets of values, but as rules for generating structure.

```
/**
 * @tparam T The type (Species) or the Function Object (Arrow).
 * @tparam Args Optional Domain for inference.
 */
export template <typename T, typename... Args>
struct SpeciesTraits;

// ...

/** @brief Universal inference for any Morphism signature f: Args... -> Codomain
```

```
 */
template <typename F, typename... Args>
struct infer_morphism {
  // For binary operators, Domain is the first argument type
  using Domain = std::tuple_element_t<0, std::tuple<Args...>>;
  using Codomain = std::invoke_result_t<F, Args...>;
};

// ...

static_assert(IsCommutative<int, std::plus<int>>,
              "Commutative: Integer addition is commutative.");

// ...
```

Listing 2: Excerpt to illustrate the anchoring the monoid operations as morphisms at compile time.

As shown in Table 4, the "Feature Cube" serves as a formal ledger mapping algebraic properties to the language's fundamental atoms. This explicit mapping allows the `dedekind` dispatcher to distinguish between strict algebraic structures (e.g., `unsigned int` addition) and numerical species, such as floating-point operations which exhibit numerical non-associativity under the IEEE 754 standard [25]. By anchoring these operators as first-class categorical entities, we enable the compiler to act as a physical guardrail for mathematical truth, ensuring structural integrity is verified prior to binary generation.

The overall intention is to anchor the C++ type system (primitive types and operators) as a substrate for modelling truths obtained from category theory and abstract algebra. In this setup, providing the axiomatic truths (e.g. $a||b$ is commutative) or the absence thereof (the messy reality of NaN, overflows, or rounding errors) is through this registry of truths the responsibility of the library developer.

Table 4: The Feature Cube: Structural Invariants for Primitive Species. **Assoc**, **Comm**, and **Idemp** denote algebraic properties of binary operations; **Refl**, **Trans**, and **Anti** denote properties of binary relations; **Period** denotes the "Safety Certificate" for totality (circular topology).

Species	**Op / Rel**	**Assoc**	**Comm**	**Idemp**	**Period**	**Refl**	**Trans**	**Anti**	**Logic**
Logical	`std::logical_`								
`bool`	`and / or`	✓	✓	✓	†	—	—	—	Class.
`bool`	`equal_to`	✓	✓	—	—	✓	✓	✓	Class.
Integral	`std::bit_`								
`uint/int`	`and / or`	✓	✓	✓	—	—	—	—	Class.
`uint/int`	`xor`	✓	✓	—	✓	—	—	—	Class.
Integral	`std::arithmetic`								
`uint`	`plus / mult`	✓	✓	—	✓	—	—	—	Class.
`int`	`plus / mult`	✓	✓	—	—	—	—	—	Class.
Floating	`std::arithmetic`								
`double`	`plus / mult`	∗	✓	—	—	—	—	—	Tern.
Relational	`std::relational`								
`uint/int`	`less_equal`	—	—	✓	—	✓	✓	✓	Class.
`double`	`less_equal`	—	—	✓	—	‡	✓	✓	Tern.

∗*Fails bit-perfect associativity due to rounding error in IEEE 754 [25].*

†*Lattice operations are total via idempotency rather than periodicity.*

‡*Fails reflexivity (`NaN <= NaN` is false).*

2.1.2 The `:logic` partition: The Subobject Classifier (Ω)

Before a "Body" (Set) or a "Path" (Sequence) can be reified, the `dedekind` library establishes the "Rules of Presence" through the **Subobject Classifier** Ω. Following the algebraic logic of Rasiowa [26], we treat every logical system as a specific type of abstract algebra where truth-values act as the objects of a **Topos**. By reifying these systems via the `LogicalSpecies` concept, we prevent the "leaky abstractions" inherent in machine-level `bool` types—such as implicit integral promotion—while allowing for pluggable logical universes.

In the `ClassicalLogic` species, Ω is the binary prime $\{True, False\}$ (see Table 5), corresponding to the standard Boolean Topos where the **Law of Excluded Middle** holds. However, to honestly model computational undecidability or the "truth-holes" inherent in floating-point NaN boundaries, we introduce the `TernaryLogic` species. Based on Kleene's strong logic of indeterminacy (K3) [19], this logic introduces an **Unknown** state (0) as seen in Table 5. Within the `dedekind` framework, these logics are formalized as **Rigs**; the *Join* ($\vee$) serves as additive composition (Supremum), while the *Meet* ($\wedge$) serves as multiplicative composition (Infimum). This structural mapping ensures that any **Predicate** $X \to \Omega$ provides the mathematically rigorous foundation required for the subobject classification in the downstream `:partial` partition.

Table 5: Truth Tables for Classical ($\Omega_{\mathbb{B}}$) and Kleene (Ω_{K3}) Toposes. In the Ternary system, the *Unknown* state (0) acts as a fixed point for negation and a neutral element for disjunction.

Table 6: *
Classical ($\Omega = \{0, 1\}$)

A	B	$A \wedge B$	$A \vee B$	$\neg A$
0	0	0	0	1
0	1	0	1	1
1	0	0	1	0
1	1	1	1	0

Table 7: *
Kleene K3 ($\Omega = \{-1, 0, 1\}$)

A	B	$A \wedge B$	$A \vee B$	$\neg A$
-1	-1	-1	-1	1
-1	0	-1	0	1
-1	1	-1	1	1
0	0	0	0	0
0	1	0	1	0
1	1	1	1	-1

The necessity of the `TernaryLogic` species is most evident when observing the breakdown of the **Law of Excluded Middle** ($P \vee \neg P = \top$) in floating-point arithmetic. Following the IEEE 754 standard [25], the `NaN` (Not-a-Number) state represents a singularity where standard relational morphisms fail to provide a binary partition of the species. Consider a predicate $P(x) \coloneqq (x \leq 1.0)$ for $x = \text{NaN}$. In classical C++, the expression `(x <= 1.0)` evaluates to `false`. However, the logical complement $\neg P(x)$, represented by `(x > 1.0)`, also evaluates to `false`. This results in the paradox $P \vee \neg P = \bot$, a violation of classical tautology that the `dedekind` library resolves by elevating the subobject classification to Ω_{K3}.

```
double x = std::numeric_limits<double>::quiet_NaN();

// In a Classical Topos (bool), both are false:
bool p     = (x <= 1.0); // false
bool not_p = (x > 1.0);  // false
bool lem   = p || not_p; // false (LEM Violation!)

// In the Dedekind Ternary Topos (Omega):
Ternary P     = Classifier<double>::eval(x <= 1.0); // Unknown
Ternary NOT_P = !P;                                 // Unknown
Ternary LEM   = P || NOT_P;                         // Unknown
```

Listing 3: The NaN Truth-Hole: Violating the Law of Excluded Middle in C++.

By mapping these "numerical hazards" to the **Unknown** state, the library ensures that downstream algebraic transformations (such as those in the `:partial` partition) do not proceed under the false assumption of a binary state. This "honesty" in the logic layer allows the compiler to treat `NaN` or integer overflow boundaries as formal subobject exclusions rather than undefined behavior.

The `dedekind` library acknowledges the machine's physical constraints by identifying two primary categories of "truth-holes" where the standard mapping between $\mathbb{Z}$ or $\mathbb{R}$ and their machine counterparts, `int` and `double`, breaks down.

The NaN Truth-Hole. As specified in the IEEE 754 standard [25], the `NaN` state represents a logical singularity. For a predicate $P(x) \coloneqq (x \leq 1.0)$, if $x = \text{NaN}$, then both $P(x)$ and its negation $\neg P(x)$ evaluate to `false`. This paradox, $P \vee \neg P = \bot$, directly violates the **Law of Excluded Middle (LEM)**, a failure that the `SubobjectClassifier` resolves by elevating the result to the **Ternary Topos** (Ω_{K3}) [19].

The Lipschitz Boundary. For signed integral types, the mapping from the infinite ring of integers to the finite machine representation is only stable within a defined "Lipschitz domain" $[-2^{N-1}, 2^{N-1} - 1]$. Once an operation, such as addition, breaches this boundary, it results in *undefined behavior* or "wrap-around" effects, effectively losing its Lipschitz continuity. By utilizing compiler built-ins (e.g., `__builtin_add_overflow`), the `SubobjectClassifier` detects these boundary breaches and maps them to the **Unknown** state, as illustrated in Listing 4.

```
export template <IsSpecies T>
struct SubobjectClassifier {
  using L = typename LogicTraits<T>::type;
  using Omega = typename L::type;

  /** @brief Lipschitz Boundary Check for Signed Integers */
  template <typename Op>
    requires std::signed_integral<T>
  static constexpr Omega evaluate_arithmetic(T a, T b, Op op) {
      T result;
      if (__builtin_add_overflow(a, b, &result)) {
          return L::Unknown; // Lipschitz boundary breached
      }
      return L::True;
  }

  /** @brief NaN Truth-Hole Check for IEEE 754 */
  template <typename Op>
    requires std::floating_point<T>
  static constexpr Omega evaluate_relational(T a, T b, Op rel) {
      if (std::isnan(a) || std::isnan(b)) {
          return L::Unknown;
      }
      return rel(a, b) ? L::True : L::False;
  }
};
```

Listing 4: Subobject Classifier for the Lipschitz Boundary and NaN Truth-Hole.

2.1.3 The `:total` partition: The Laws of Harmony

In the "Platonic" domain of our ontology, we define the category $\mathcal{C}$ of **Total Species**, where algebraic structures are governed by a collection of total morphisms $\mu : X \times X \to X$. Following the *Polish School of Logic* [26], these structures represent "Laws of Harmony" that are treated as absolute invariants. This approach aligns with the *Elementary Theory of the Category of Sets* (ETCS) as proposed by Lawvere [27, 21], where types are defined by their morphisms rather than their internal elements [28].

As shown in Table 8, species in this partition "mature" as they gain axioms, moving from the primal **Magma** to the perfect symmetry of an **Abelian Group**. A primary inhabitant of this total universe is the `bool` species; under the morphisms of logical conjunction ($\wedge$) and disjunction ($\vee$), it behaves as a **Distributive Lattice**—a specialized **Rig** where both operations are **Idempotent**. This idempotency provides the structural basis for **Universal Constructions** such as *Products* and *Coproducts*, which appear in systems programming as *Meet* and *Join* operations [13].

By mapping these structures to C++23 concepts, the `dedekind` library transforms the compiler into a **Constraint Satisfaction Engine** (CSE) that verifies mathematical truth as a physical invariant of the binary [29]. This leads to the **Honest Rejection** illustrated in Listing 5: while `unsigned int` addition is verified as a total Abelian Group ($\mathbb{Z}/2^n\mathbb{Z}, +$), signed `int` addition is rejected as a total Magma due to the potential for overflow at the Lipschitz boundary.

Table 8: The Path to Symmetry: Single-Operation Species in `dedekind.category`. Each level introduces a new structural invariant, from basic closure to commutative inversion.

Concept	Structure	Axiomatic Requirements	C++ Concept
Magma	$\langle T, \circ \rangle$	$\forall a, b \in T : a \circ b \in T$ (Totality)	`IsMagma`
Semigroup	$\langle T, \circ \rangle$	Magma + Associativity	`IsSemigroup`
Monoid	$\langle T, \circ, e \rangle$	Semigroup + Identity	`IsMonoid`
Group	$\langle T, \circ, e, \text{inv} \rangle$	Monoid + Inverse	`IsGroup`
Abelian Group	$\langle T, \circ, e \rangle$	Group + Commutativity	`IsAbelianGroup`

```
// Level 1.3: The Perfect Symmetry (Monoid + Inverse + Symmetry)
export template <typename T, typename Op>
concept IsAbelianGroup = IsMonoid<T, Op> &&
                         IsLoop<T, Op>   &&
                         IsCommutative<T, Op>;

/** @section Truth_Anchors */

// SUCCESS: Unsigned addition is a Total Abelian Group (Z/2^nZ).
static_assert(IsAbelianGroup<unsigned int, std::plus<unsigned int>>);

// HONEST REJECTION: Signed addition is NOT a Total Magma.
// It fails because of Undefined Behavior/Overflow (Partiality).
static_assert(!IsMagma<int, std::plus<int>>);
```

Listing 5: Structural Maturation and the Honest Rejection in `dedekind.category`.

2.1.4 The `:partial` partition: The Pragmatic Rescue

The `:partial` partition performs the "Categorical Rescue" of species that failed the strict requirements of the `:total` domain. This "Rescue Mission" is where we formalise the transition from the *Platonic* to the *Pragmatic*: the goal is to show that while the machine is "broken," it is **consistently broken**. Following the informal explorations of the *n-Category Café* [30] and the subsequent formalisations of the *nLab* community [31], we acknowledge these consistent failures as structured data.

By utilizing the **Ternary Topos** (Ω_{K3}), the library "clips" the domain of definition to exclude singularities such as NaN and arithmetic overflow. To enable composition, these partial functions are lifted into the **Kleisli Category** of the *Maybe Monad* [32]. This "Rescue" ensures that even when we navigate the messy reality of hardware, we remain within a rigorous algebraic landscape, where the "Rules of Thought" are preserved even as the "Laws of Harmony" are relaxed.

This "Rescue Mission" is where we formalise the transition from the *Platonic* to the *Pragmatic*: the goal is to show that while the machine is "broken," it is **consistently broken**. By acknowledging these consistent failures—such as the Lipschitz boundary of signed integers or the truth-holes of IEEE 754—we can build a **Kleisli Category** that remains algebraically rigorous despite the physical limitations of the hardware.

Following topos-theoretic foundations [19], we treat a partial morphism $f : A \rightharpoonup B$ as a total mapping from a *subobject* $S \hookrightarrow A$, where S is the **Support** defined by the characteristic function $\chi : A \to \Omega_{K3}$ provided by the `:logic` partition. Within this restricted domain, we can define *locally total* structures, such as **Partial Groups** or **Partial Loops**. These structures guarantee that the laws of harmony hold strictly on the support $U = \{a \in A \mid \chi(a) = \top\}$.

To enable composition, these partial functions are lifted into the **Kleisli Category** of the *Maybe Monad* [32]. As codified by the *n-Lab* community [31], this category allows us to treat "effectful" or partial computations as regular morphisms by shifting the composition logic into the monad itself. In our implementation, the composition $g \circ f$ automatically propagates the **Unknown** state: if an operation breaches the Lipschitz boundary or encounters a NaN singularity, the entire chain "monadically" transitions to indeterminate. This prevents the execution of undefined machine behavior while maintaining the structuralist integrity of the transformation pipeline.

```
// The "Rescue": int addition is recognized as a Partial Abelian Group.
// It is total only on the subobject where overflow is absent.
static_assert(IsPartialAbelianGroup<int, std::plus<int>>);

// Composition in the Kleisli Category:
auto f = partial_plus(1, 2);          // Result: 3 (True)
auto g = partial_plus(f, INT_MAX);    // Result: Unknown (Overflow)
auto h = partial_plus(g, 1);          // Result: Unknown (Propagation)
```

Listing 6: The Kleisli Lift: Reclaiming the Lipschitz Boundary via Subobject Exclusion.

2.1.5 The ':numeric' partition

Finally, we encounter the "Curious" case of *Numerical Species* (IEEE 754). These types are categories where the subobject classifier Ω is internalized within the species itself (`NaN`, `Inf`). While these structures appear to be total morphisms $\mu : \mathbb{R}_f \times \mathbb{R}_f \to \mathbb{R}_f$ in their signature, they are semantically partial. Specifically, they fail the **Equalizer** conditions required for associativity; the rounding error ensures that $(a + b) + c$ and $a + (b + c)$ are not generally isomorphic. This progression—from the absolute (*Total*), through the filtered (*Partial*), to the exceptional (*Numerical*)—allows us to build a "strictly speaking" true ontology that respects both categorical rigor and machine-level reality.

2.1.6 The Semantic Mapping

The `dedekind.category` module serves as an anchor of the DSL in the established formalism of category theory. It aggregates skeletal, mereological, and algebraic laws to enable the synthesis of the *Continuum* from the *Discrete*. The adopted *structuralist* posture views mathematical objects as positions within a structure rather than isolated collections. McLarty [19] demonstrated that this is sufficient for the complete recovery of number theory. In turn, we facilitate formal verification within the C++ type system with zero runtime overhead.

Note the pain with C++ "transparent" operator mappings in the truth registry.

The stratification of the `dedekind.ontology` into C++ module partitions mirrors the constructive dependencies of formal mathematics. At the foundation, an embedding of Category Theory (Level 0) within the type system provides the primary substrate; as it makes the fewest assumptions—treating objects merely as nodes in a graph of morphisms—it establishes the "highways" (Functors and Kleisli Triples) necessary for all subsequent synthesis. Because higher algebraic structures—such as Groups, Rings, and Fields—are formally defined by the operations performed upon sets, the build order mandates that Level 1 (Space and Mereology) precedes Level 3 (Algebra). This ensures that the "Soul" of the system (its laws of action and harmony) is always grounded in the "Body" (the structural presence of its parts). By the time the compiler reaches the `:algebra` partition, the mereological validity of the underlying species is already a type-checked invariant within the translation unit. This hierarchy transforms the C++ module mapper into a directed acyclic graph of mathematical truth, where the linker cannot even begin its work until the ontological prerequisites are satisfied. The strict directed acyclic graph (DAG) of the dedekind module partitions mirrors the Bourbaki style of 'Mother Structures' [24], where the physical order of compilation serves as a proof of logical consistency.

2.2 `sets` Module

For the set operations, we may also want to look at: "Monadic Intersection Types, Relationally, and Ordered https://dl.acm.org/doi/10.1145/3777483 ... and references cited therein.

2.3 `order` Module

2.4 `topology` Module

2.5 `sequences` Module

2.6 `algebra` Module

Needed to push back repeatedly on attempts to go the convenient 'double' route instead of spelling things out as semi-modules, semi-rings etc. i.e. reduced-strength algebraic structures. Of course, ultimately everything is a complex number from this perspective just like everything is a string for front-end developers.

2.7 `morphologies` Module

2.8 `geometry` Module

2.9 `numbers` Module

2.10 `analysis` Module

The physical build order of the `dedekind.ontology` module mirrors this conceptual hierarchy. By enforcing a linear dependency graph—where `:mereology` is compiled before `:algebra`—we ensure that the compiler's proof-search for a Ring's identity element is grounded in the prior verification of its mereological subobjects. This hierarchy transforms the C++ module mapper into a directed acyclic graph of mathematical truth.

2.11 The Compiler as a Deterministic Proof Assistant

The core of `dedekind` is a recursive template evaluator that treats C++ `concepts` as logical predicates. Unlike traditional template metaprogramming that relies on SFINAE, we utilize C++23's `requires` expressions to perform "structural inspection" of types.

In the `dedekind` framework, the C++ compiler acts as a decidability engine for our mathematical ontology. Rather than relying on traditional class hierarchies, we use C++23 Concepts to define the structural requirements of a species. A type is not explicitly inherited from a base; instead, it is verified to be compliant with a species' `concept` through its intrinsic properties.

This verification is supported by Template Specialization to define axiomatic base cases and SFINAE to prune invalid overloads. We leverage the Curiously Recurring Template Pattern (CRTP) within our internal wrappers to provide a unified interface for disparate types without the overhead of dynamic dispatch. By conducting these checks in a `constexpr` context and utilizing Move Semantics for resource management, we ensure that the "Proof" of a mathematical construction is resolved entirely at compile-time.

```
// Verifying that the intersection of disjoint sets is empty
using Evens = dedekind::set<int, [](auto x){ return x % 2 == 0; }>;
using Odds  = dedekind::set<int, [](auto x){ return x % 2 != 0; }>;

using Intersection = dedekind::meet_t<Evens, Odds>;

// The compiler resolves this to the Empty Set species
static_assert(dedekind::is_empty_v<Intersection>);
static_assert(sizeof(Intersection) == 1); // No machine state required
```

Listing 7: LCompile-time inference of set invariants

2.12 The Semantic Mapping

Crucially, the `dedekind.ontology` acts as the definitive mapping between formal mathematical nomenclature and the concrete syntax of the C++23 language. Rather than introducing a foreign vocabulary, we anchor established axioms directly into the standard operator set and primitive types. For instance, the mereological *Sum* and *Product* are reified as the bitwise operators `|` and `&`, while the *Mereological Complement* (the remainder) is bound to the negation operator `!`.

By establishing this explicit correspondence—where `operator<=` serves as the formal proof of *Parthood* ($\sqsubseteq$) and `operator/` represents the *Quotienting* of a species—we ensure that the resulting code is not merely a simulation of a mathematical model, but a direct execution of it. This semantic alignment allows the programmer to write expressions that are readable as equations, while the compiler treats them as a sequence of verifiable type-transformations. In this architecture, the C++ `concept` becomes the "Gatekeeper of Truth," ensuring that a species cannot participate in an operation (such as a Join-Semilattice union) unless it has first provided a manual or structural certification of its algebraic invariants.

2.13 The Registry of Structural Proofs

The transition from a raw computational graph to a simplified mathematical identity requires a formal "Seal of Truth." In `dedekind`, we implement a *Traits Registry* that forces the developer to explicitly certify the equational axioms of a species before it can participate in the lattice-based optimizations of the dispatcher.

This is achieved through a dual-layered system of **Trait Ledgers** and **Discovery Concepts**. For each fundamental algebraic law, we define a primary template (the ledger) which defaults to `false`:

```
export template <typename T, typename Op>
struct is_associative : std::false_type {};

export template <typename T, typename Op>
inline constexpr bool is_associative_v = is_associative<T, Op>::value;
```

Listing 8: The Associativity Ledger in :species

Table 9: The Dedekind Semantic Mapping: A Registry of Formal Correspondences

Mathematical Concept	**Symbol**	**C++ Operator / Type**	**C++ Concept**	**Basis**
Category Theory				
Kleisli Extension	$(T, \eta, \gg=)$	operator>>=	IsKleisliExtension	IsAssociative
Kleisli Extension	$\gg=$	operator>>=	IsKleisliExtension	IsAssociative
Kleisli Composition	$\circ\_K$	operator>>	IsMonad	IsAssociative
Co-Kleisli Extension	$\gg=_{co}$	operator<<=	IsComonad	IsAssociative
Morphic Injection (Unit)	η	into<F> / singleton	IsMonad	$\eta : T \to F(T)$
Morphic Extraction (Counit)	ε	extract<F>	IsComonad	$\varepsilon : F(T) \to T$
Characteristic Morphism	χ	operator()	IsCharacteristic	
Co-Kleisli Composition	$\circ\_CoK$	operator<<	IsComonad	
Counit (Extraction)	ε	extract<F>	IsComonad	
Mereology				
Proper Parthood	χ	operator()	IsProperPart	IsCharacteristic
Parthood (Pre-order)	$\sqsubseteq$	operator<=	IsPartOf	
Mereological Sum (Join)	$\sqcup$	operator|	IsJoinSemilattice	
Mereological Product (Meet)	$\sqcap$	operator&	IsMeetSemilattice	
Universal Complement	$\neg$	operator!	IsComplemented	
Initial Object (Empty)	$\emptyset$	∅<T>	IsInitialObject	
Terminal Object (Universal)	Ω	Ω<T>	IsTerminalObject	
Set Theory				
Membership	$\in$	operator()	IsSet	IsProperPart
Subset	$\subseteq$	operator<=	IsSubset	IsPartOf
Countable Infinity	$\aleph_0$	ℵ_0	IsCardinality	
Continuum Cardinality	$\beth_1$	ℶ_1	IsCardinality	
Order Theory				
Pre-order	$\sqsubseteq$	operator<=	IsPartiallyOrdered	IsPartOf
Algebra				
Structural Origin (Zero)	0	T::origin()	IsPointed	
Additive Composition	$+$	operator+	IsGroup	
Additive Inverse	$-$	operator-	IsGroup	
Multiplicative Composition	$\times$	operator*	IsRing	
Multiplicative Inverse	$/, ^{-1}$	operator/	IsField	
Associativity	$(a \circ b) \circ c$	is_associative_v	IsSemigroupoid	
Idempotency	$a \circ a = a$	is_idempotent_v	IsIdempotent	
Magnitude / Count	$\lvert S \rvert$	s.size() / s.cardinality()	IsCardinality	
Supremum / Infimum	$\sup, \inf$	s.supremum(), s.infimum()	HasExtrema	

A species S is only recognized as a `IsSemigroupoid` if it (or the ontology) provides a specialization of this ledger. To allow for "Interoperable Discovery," we provide a fallback mechanism that probes the species for internal certification:

```
export template <typename T, typename Op>
concept IsAssociative = IsMagmoid<T, Op> && requires {
    requires is_associative_v<T, Op>;
};

// Discovery Fallback: Probing the Species for an Intrinsic Proof
template <typename T, typename Op>
  requires requires { T::is_associative_for<Op>; }
struct is_associative<T, Op> : T::template is_associative_for<Op> {};
```

Listing 9: The Concept of Formal Association

By anchoring concepts like `IsJoinSemilattice` in these requirements (requiring both `IsAssociative` and `IsIdempotent`), we ensure that the *Lattice Dispatcher* never performs a "speculative" pruning. If the developer has not provided the proof, the dispatcher falls back to a safe, unoptimized `UnionSet` synthesis. This "Axiom of Responsibility" ensures that the performance of the "Naked Loop" is always mathematically justified by a prior formal certification.

2.14 Ontological Model Validation

To verify the integrity of the proposed mapping between formal axioms and C++ concepts, we employ a "Reverse Curry-Howard" validation strategy. Rather than simply assuming the correctness of our ontological translations, we use the inherent properties of C++ built-in types as an empirical proof of the reverse-engineered axioms.

By executing a suite of `static_assert` evaluations on primitive types—such as verifying that `bool` satisfies the requirements of an `IsJoinSemilattice` or that bitwise operations on `uint32_t` adhere to the `IsMereologicalLattice` consistency axioms—we confirm that the machine-level behavior is isomorphic to the formal mathematical theory. In this posture, the built-in types serve as the "Initial Models" for our concepts. If the compiler successfully verifies that a standard integer under bitwise conjunction satisfies our `IsMeetSemilattice` proof, it provides an automated, formal confirmation that our C++ concepts correctly capture the essential structural invariants found in the mathematical literature.

3 Implementation: The Functor Highway

The operational efficiency of `dedekind` is achieved through a technique we term *Synthetic Inference*. This level represents the "Skeletal Foundation" of the ontology, where machine-level instructions are unified with the abstract laws of Category Theory. By treating the C++ type system as a formal proof assistant, we ensure structural integrity via compile-time verification, or *Static Mereology*.

3.1 The Action-First Bootstrapping Strategy

In textbook Category Theory, a Monad is traditionally defined as a *Monoid in the category of Endofunctors*. However, the structural implementation of this definition in C++ faces significant language constraints:

1. **Partial Specialization Limitations**: C++ forbids the partial specialization of function templates (e.g., `fmap<F>`), preventing a centralized, generic definition for disparate species.

2. **ADL Resolution**: Function templates called via explicit name-lookup cannot trigger Argument-Dependent Lookup (ADL), making it difficult for the ontology to "discover" mappings for types defined in downstream modules.

To resolve this circular dependency, `dedekind` implements an *Action-First* bootstrapping strategy. By defining Monads and Comonads as *Extension Systems* (Kleisli Triples consisting of η/ε and $\gg=$ / $\ll=$), we utilize operator overloading on concrete objects. This triggers ADL at the call site, allowing the Level 0 foundation to instantiate a derived `fmap` without prior knowledge of Level 1 species.

3.2 The Unified Highway Bridge

The `fmap` dispatcher serves as the implementation of this strategy. It automates the derivation of functorial mappings from the underlying monadic "Push" or comonadic "Pull."

```
export template <template <typename...> typename F, typename Arrow,
                 typename T = typename Arrow::Domain,
                 typename U = typename Arrow::Codomain>
  requires IsArrow<Arrow, T, U>
constexpr IsArrow<F<T>, F<U>> auto fmap(Arrow f) {
  if constexpr (std::same_as<F<T>, T>) {
    return f; // Identity Functor
  } else if constexpr (IsKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = m >>= (eta o f) */
    return arrow<F<T>, F<U>>([f](const F<T>& m) {
      return m >>= [f](const T& x) { return into<F, U>(f(x)); };
    });
  } else if constexpr (IsCoKleisliExtension<F, T, U>) {
    /** @theorem fmap(f) = w <<= (f o epsilon) */
    return arrow<F<T>, F<U>>([f](const F<T>& w) {
      return w <<= [f](const F<T>& ctx) { return f(extract<F, T>(ctx)); };
    });
  } else {
    static_assert(sizeof(T) == 0, "Ontology Error: Species lacks highway structure."
  }
}
```

Listing 10: The Unified Highway Bridge (fmap derivation)

3.3 Highway Notation: Directional Vectors

To facilitate a readable "Algebra of Programming," we map categorical data flows to directional operators, creating a "Highway Notation" that reflects the vector of computation across the ontology:

- `value » into<F>` : Represents η (Unit), lifting a species into a context.
- `box « extract<F>` : Represents ε (Counit), sampling a species from a context.
- `box »= f` : Represents *Bind*, the forward monadic composition (Left-to-Right).
- `box «= f` : Represents *Extend*, the contextual comonadic composition (Right-to-Left).

```
// Convert plain lambdas into tagged endomorphisms:
auto f = endo<int>([](int x) { return x + 1; });
auto g = endo<int>([](int x) { return x * 2; });
```

```
// Morphism composition: (h = g . f):
auto h = f >> g;

// Lifting into the Box and applying the composed morphism:
auto result = 5 >> into<Box> >> fmap<Box>(h);

CHECK(result == Box<int>{12});
```

Listing 11: Example of fused Kleisli "Highway" composition via the functor law.

```
// Define two monadic arrows (Species -> Boxed Species)
auto f = [](int x) { return pure(x + 1); };
auto g = [](int x) { return pure(x * 2); };

// Chain composition via Bind (>>=)
// Starting with a raw value, lifting it, then chaining the monadic arrows.
auto x = (5 >> into<Box> >>= f) >>= g;

// (5 + 1) * 2 = 12, wrapped in a Box
CHECK(x == Box<int>{12});

// Verification of the Monadic Associativity Law:
// (m >>= f) >>= g  is equivalent to  m >>= (x => f(x) >>= g)
auto lh = (5 >> into<Box> >>= f) >>= g;

auto rh = 5 >> into<Box> >>= [&](int x) { return f(x) >>= g; };

CHECK(lh == rh);
STATIC_CHECK(std::same_as<decltype(x), Box<int>>);
```

Listing 12: Chain Composition (Bind / »=)

This mapping ensures that the "Categorical Cement" holding the "Algebraic Bricks" together is both syntactically expressive and computationally invisible.

4 Implementation: The Compile-Time Logic Engine

The core of `dedekind` is a recursive template evaluator that treats C++ `concepts` as logical predicates. Unlike traditional template metaprogramming that relies on SFINAE, we utilize C++23's `requires` expressions to perform "structural inspection" of types.

4.1 The Compiler as a Deterministic Proof Assistant

In the `dedekind` framework, the C++ compiler acts as a decidability engine for our mathematical ontology. Rather than relying on traditional class hierarchies, we use C++23 Concepts to define the structural requirements of a species. A type is not explicitly inherited from a base; instead, it is verified to be compliant with a species' `concept` through its intrinsic properties. This allows for a structuralist approach where any type—primitive or user-defined—can participate in the mereology if it satisfies the necessary algebraic invariants. This verification is supported by Template Specialization to define axiomatic base cases and SFINAE to prune invalid overloads. We leverage the Curiously Recurring Template Pattern (CRTP) within our internal wrappers to provide a unified interface for disparate types without the overhead of dynamic dispatch. By conducting these checks in a `constexpr` context and utilizing Move Semantics for resource management, we ensure that the "Proof" of a mathematical construction is resolved entirely at compile-time, leaving only optimized machine code.

```
// Verifying that the intersection of disjoint sets is empty
using Evens = dedekind::set<int, [](auto x){ return x % 2 == 0; }>;
using Odds = dedekind::set<int, [](auto x){ return x % 2 != 0; }>;
using Intersection = dedekind::meet_t<Evens, Odds>;
// The compiler resolves this to the Empty Set species
static_assert(dedekind::is_empty_v);
static_assert(sizeof(Intersection) == 1); // No machine state required
```

Listing 13: Compile-time inference of set invariants

```
// In dedekind, a valid program is a constructive proof.
// If the 'Proof' (the type-transformation) fails, the 'Program' does not compile.
template
requires dedekind::is_archimedean_field_v
auto compute_limit(T sequence) {
return sequence.limit();
}
// Attempting to compute a limit on a Discrete Ring:
// The compiler acts as a proof-assistant and rejects the invalid mathematical path
// Error: constraints not satisfied for 'is_archimedean_field_v'
auto result = compute_limit(my_discrete_ring);
```

Listing 14: The Reverse Curry-Howard Isomorphism in action

4.2 Reifying the Set-Builder

To achieve the syntax

$$\mathrm{Set}\{\mathrm{x} \,\%\, \mathbb{R} \wedge (\mathrm{x} + 5 < 10)\} \tag{6}$$

, we introduce a `set` template that accepts a domain and a predicate. The predicate is not a function pointer, but a *Stateless Constraint Type*.

```
import dedekind.numbers;
import dedekind.logic;
// Define a predicate: x is even and greater than 10
auto P = [](auto x) {
return (x % 2 == 0) && (x > 10);
};
// Materialize the set at compile-time
using EvenGreaterTen = dedekind::set<int, P>;
static_assert(dedekind::contains(12));
static_assert(!dedekind::contains(7));
```

Listing 15: Example of Set-Builder realization in Dedekind

4.3 The Role of C++23 Modules

The stratification mentioned in Section 2 is strictly enforced via the module system. By using `export module dedekind.ontology:mereology`, we ensure that the compiler can cache the "structural truths" of the system. This prevents the exponential blow-up of compile times typically associated with heavy template libraries, as the "laws" of the ontology are compiled once into binary module interfaces (BMIs)

4.4 Continuous Integration

The implementation of the `dedekind` library and its associated build graph are available on GitHub [29]. We utilize GitHub Actions to verify the project's structural integrity; each commit is checked via `static_assert` compile-time proofs and a suite of automated tests. Following the tradition of property-based testing [10], we utilize these tests as "runtime witnesses" to verify the materialization of our categorical primitives, aiming for a patch coverage threshold of 60% or higher for all new module partitions. While the automation of build and test cycles is considered a 'bread-and-butter' practice in contemporary software engineering [33], its application in the dedekind framework serves a specific epistemological purpose. By treating the CI pipeline as a continuous verification engine, we ensure that the categorical ontology remains decidable and physically materialized across all translation units.

5 Conclusion

This paper has explored the feasibility of embedding formal mereology within the C++23 type system through the `dedekind` library. By transitioning from the traditional object-oriented paradigm of information hiding toward a model of structural transparency, we have illustrated how the compiler can be utilized as a rudimentary constraint satisfaction engine. This approach allows for the high-level representation of mathematical species, such as set-builder notation, without the typical runtime performance penalties associated with abstract modeling.

Our investigation suggests that while the Clang/LLVM toolchain is not a dedicated theorem prover, it can effectively implement a 'rethought' set theory [34], where categorical axioms provide the semantic context for aggressive optimization. By hoisting mereological relations into the type signature, we enable the compiler to perform "structural pruning"—collapsing contradictions and optimizing intersections—resulting in machine code that maintains the efficiency of hand-tuned assembly.

The stratification of these formalisms into module partitions provides a scalable pathway for managing the "template tax," ensuring that the rigors of formal logic do not destabilize the build pipeline. By embedding the Dedekind cut and the construction of the Continuum into the build process, we have moved toward a nascent paradigm of *Axiomatic Systems Programming*: a development style where the build process acts as a physical guardrail for mathematical truth.

Future work will investigate extending this mereological framework to other domains of the computational sciences and high-performance computing, such as a direct integration with the upcoming linear algebra capabilities of the C++ standard library [35] while expressly facilitating computations not just using floating point numbers but more generally supporting simplified algebraic systems such as semimodules over rings ($\mathbb{B}, \mathbb{N}$) or extended number systems such as the complex numbers ($\mathbb{C}$), the dual numbers or quaternions.

References

[1] Adam Paszke, Sam Gross, Francisco Massa, et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library". In: *Advances in Neural Information Processing Systems 32*. Ed. by H. Wallach et al. 2019, pp. 8024–8035. URL: `https://neurips.cc`.

[2] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. "Array programming with NumPy". In: *Nature* 585.7825 (2020), pp. 357–362. DOI: `10.1038/s41586-020-2649-2`.

[3] Wenzel Jakob. *nanobind: tiny and efficient C++/Python bindings*. `https://github.com/wjakob/nanobind`. 2022.

[4] Wim T. L. P. Lavrijsen and Aditi Dutta. "High-Performance Python-C++ Bindings with PyPy and Cling". In: *Proceedings of the 15th Python in Science Conference (SciPy 2016)*. Ed. by Sebastian Benthall and Scott Rostrup. 2016, pp. 27–35. DOI: `10.25080/Majora-629e541a-004`.

[5] ISO/IEC JTC 1/SC 22. *ISO/IEC 14882:2024: Programming languages — C++*. Commonly referred to as C++23. International Organization for Standardization. Geneva, Switzerland, 2024. URL: `https://www.iso.org`.

[6] Bartosz Milewski. *Category Theory for Programmers*. An excellent bridge between C++ type systems and categorical structures. Blending Science, 2018.

[7] Stewart Shapiro. *Thinking about Mathematics: The Philosophy of Mathematics*. See Part IV for an accessible introduction to structuralism. Oxford University Press, 2000.

[8] The GHC Development Team. *The Glasgow Haskell Compiler User's Guide, Edition 2024*. GHC 2024 Language Edition. 2024. URL: `https://downloads.haskell.org/ghc/latest/docs/users_guide/exts/control.html`.

[9] Ulf Norell. "Towards a practical programming language based on dependent type theory". PhD thesis. Chalmers University of Technology, 2007. URL: `http://www.cse.chalmers.se`.

[10] Koen Claessen and John Hughes. "QuickCheck: a lightweight tool for random testing of Haskell programs". In: *Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP)*. ACM, 2000, pp. 268–279.

[11] Bryan O'Sullivan, John Goerzen, and Don Stewart. *Real World Haskell*. O'Reilly Media, 2008.

[12] Steve Awodey. "An Answer to Hellman's Question: Does Category Theory Provide a Framework for Mathematical Structuralism?" In: *Philosophia Mathematica* 12.1 (2004), pp. 54–64.

[13] Bartosz Milewski. *Category Theory for Programmers*. Compiled from the original blog series at `https://bartoszmilewski.com`. Blurb, Incorporated, 2019. ISBN: 9780464243878.

[14] F. William Lawvere and Robert Rosebrugh. *Sets for Mathematics*. Cambridge University Press, 2003.

[15] André Joyal. "Une théorie combinatoire des séries formelles". In: *Advances in Mathematics* 42.1 (1981), pp. 1–82. DOI: `10.1016/0001-8708(81)90052-9`.

[16] André Joyal. "Fonctions analytiques et espèces de structures". In: *Combinatoire énumérative*. Springer, 1986, pp. 126–159. DOI: `10.1007/BFb0072514`.

[17] F William Lawvere. "Metric spaces, generalized logic, and closed categories". In: *Rendiconti del seminario matematico e fisico di Milano* 43.1 (1973), pp. 135–166.

[18] Francis Borceux and Françoise Dejean. "Cauchy completion in category theory". In: *Cahiers de topologie et géométrie différentielle catégoriques* 27.2 (1986), pp. 133–146.

[19] Colin McLarty. *Elementary Categories, Elementary Toposes*. Clarendon Press, 1992.

[20] Thomas Mormann. "Structural Mereology and Statistics". In: *Logic and Logical Philosophy* 22.4 (2013), pp. 427–453.

[21] F William Lawvere. "An elementary theory of the category of sets (long version) with commentary". In: *Reprints in Theory and Applications of Categories* 11 (2005), pp. 1–35.

[22] Joachim Lambek and Philip J Scott. *Introduction to Higher-Order Categorical Logic*. Cambridge University Press, 1988.

[23] nLab authors. *structural set theory*. ncatlab.org/nlab/show/structural+set+theory. 2024. URL: `https://ncatlab.org`.

[24] Jean-Pierre Marquis. "The Structuralist Mathematical Style: Bourbaki as a Case Study". In: *The Architecture of Modern Mathematics*. Oxford University Press, 2022.

[25] IEEE. *IEEE Standard for Floating-Point Arithmetic*. Provides the formal specification for floating-point formats and operations. 2019. DOI: 10.1109/IEEESTD.2019.8766229.

[26] Kazimierz Ajdukiewicz. *Język i Poznananie (Language and Cognition)*. Foundational text for the Polish School of Logic and Semantic Epistemology. Państwowe Wydawnictwo Naukowe, 1985.

[27] F. William Lawvere. "An elementary theory of the category of sets". In: *Proceedings of the National Academy of Sciences* 52.6 (1964), pp. 1506–1511.

[28] Colin McLarty. "Numbers can be just what they have to". In: *Noûs* 27.4 (1993), pp. 487–498.

[29] Vincent Kräutler. *dedekind: A Structuralist Mereology in C++23*. Version v1.0.0. 2026. URL: https://github.com.

[30] John Baez, Urs Schreiber, and David Corfield. *The n-Category Café: A group blog on math, physics and philosophy*. https://utexas.edu. 2006–present.

[31] nLab authors. *Kleisli category*. https://ncatlab.org. Accessed: 2026-04-04. 2026.

[32] Heinrich Kleisli. "Every standard construction is induced by a pair of adjoint functors". In: *Proceedings of the American Mathematical Society* 16.3 (1965), pp. 544–546. DOI: 10.2307/2034658.

[33] Paul M. Duvall, Steve Matyas, and Andrew Glover. *Continuous Integration: Improving Software Quality and Reducing Risk*. Signature Series. Addison-Wesley Professional, 2007.

[34] Tom Leinster. "Rethinking set theory". In: *The American Mathematical Monthly* 121.5 (2014). arXiv:1212.6543, pp. 403–415.

[35] Mark Hoemmen et al. *P1673R13: A free function linear algebra interface based on the BLAS*. Tech. rep. ISO/IEC JTC1/SC22/WG21, 2024. URL: https://wg21.link.